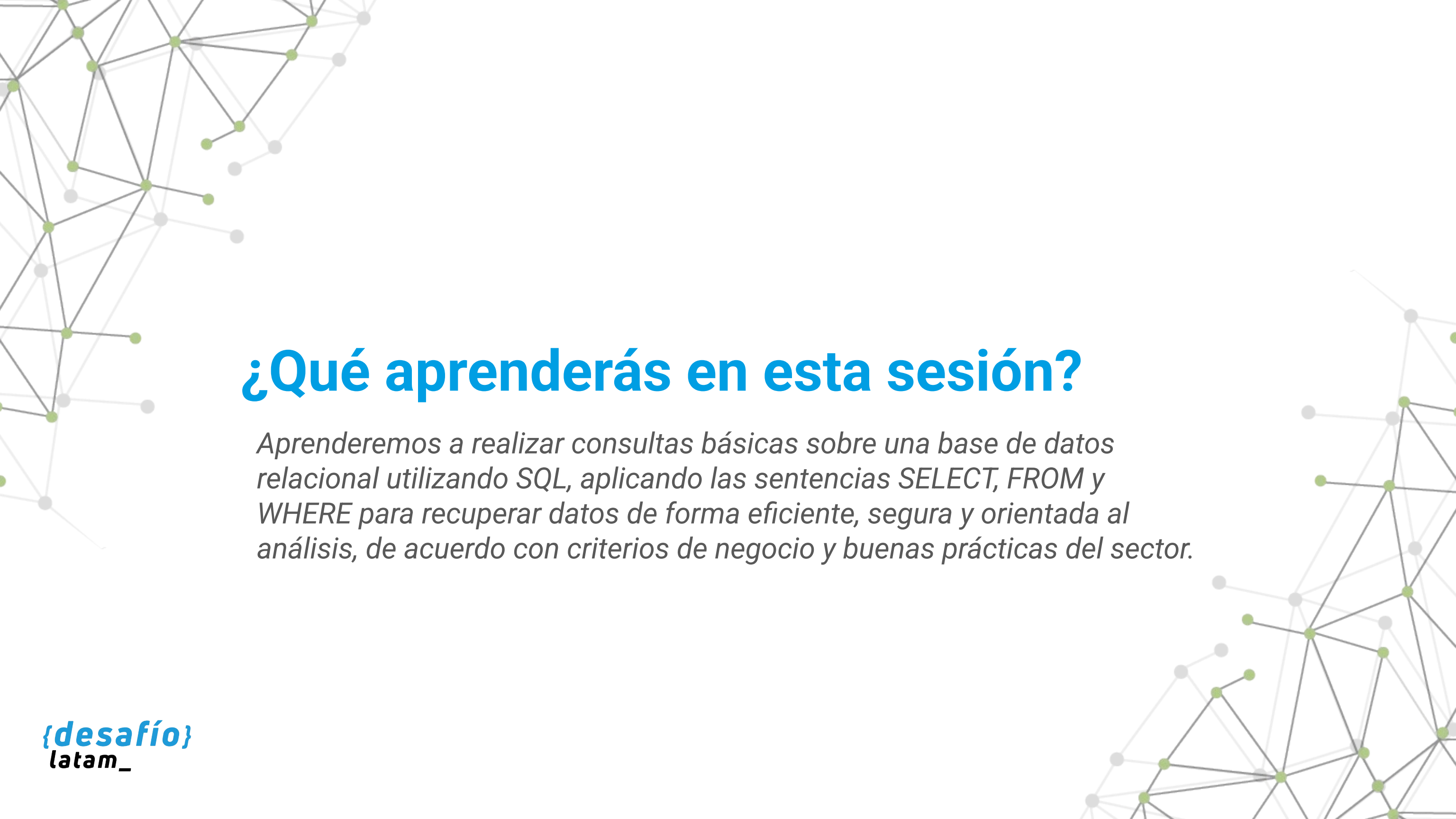




Fundamentos de Bases de Datos y Consultas Básicas

Clase 4 - Consultas básicas con SELECT



¿Qué aprenderás en esta sesión?

Aprenderemos a realizar consultas básicas sobre una base de datos relacional utilizando SQL, aplicando las sentencias SELECT, FROM y WHERE para recuperar datos de forma eficiente, segura y orientada al análisis, de acuerdo con criterios de negocio y buenas prácticas del sector.

Desafío inicial

Eres parte del equipo de inteligencia de negocios de una empresa de logística que está implementando un tablero de control para supervisar sus operaciones en tiempo real. El equipo directivo necesita obtener rápidamente reportes filtrados por zona geográfica, tipo de envío y estado de la entrega. Hasta ahora, todo se hacía manualmente en Excel, pero tú deberás demostrar cómo automatizar la consulta de estos datos desde la base de datos relacional.

Tienes acceso a la base de datos, pero necesitas saber:

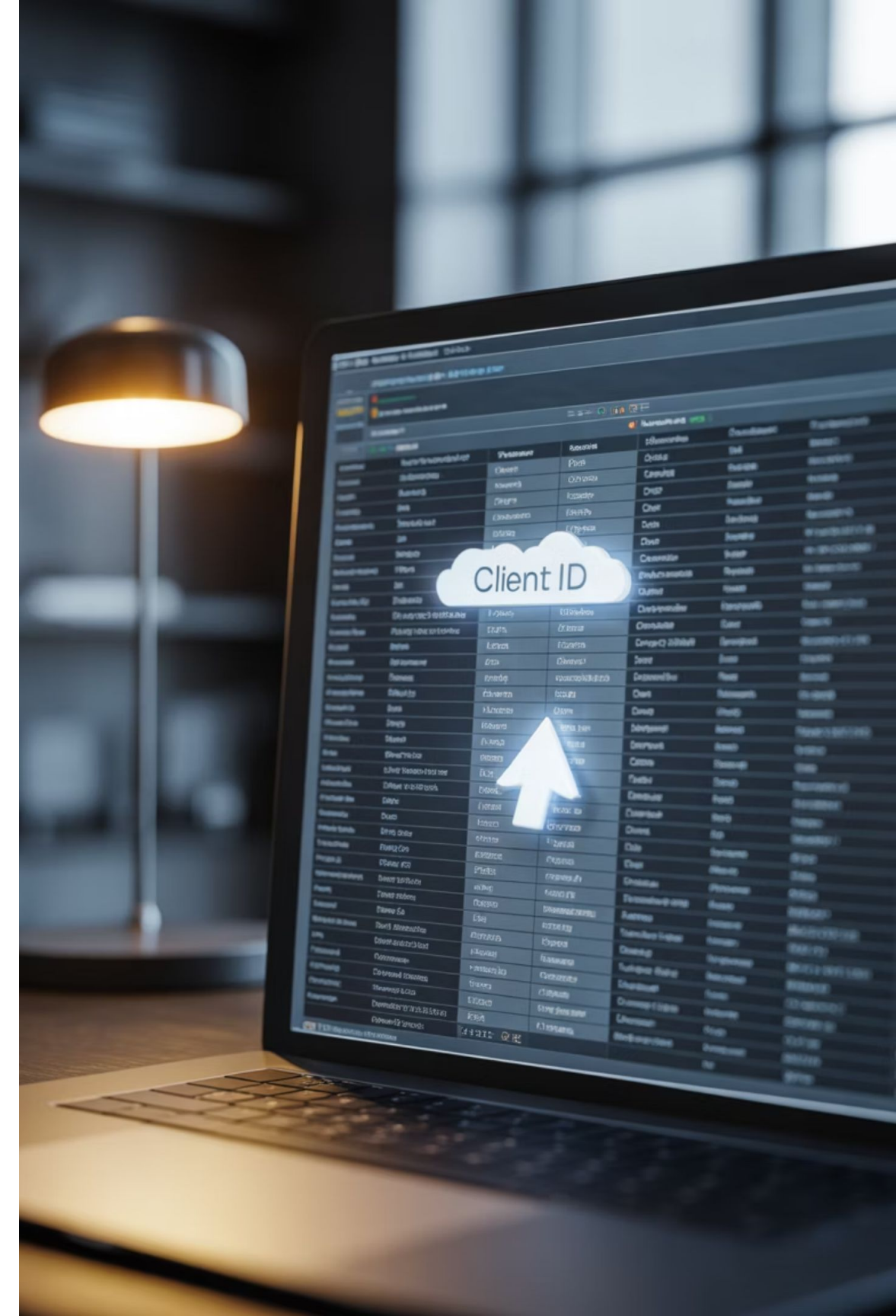
- ¿Cómo seleccionar solo las columnas que deben mostrarse en el informe?
- ¿Desde qué tabla conviene extraer los datos?
- ¿Qué condiciones debes aplicar para limitar la consulta a entregas realizadas durante el último mes, en ciudades específicas, y que aún estén "en tránsito"?

Durante esta sesión, aprenderás a escribir consultas SQL que te permitan resolver este desafío y entregar al equipo un conjunto de resultados filtrado y preciso, con los campos adecuados para ser usados en informes, dashboards y reportes ejecutivos.

Selección de columnas con SELECT

¿Cómo podrías consultar solo la información específica que necesitas sin sobrecargar tu resultado con columnas irrelevantes?

Al trabajar con bases de datos relacionales, una de las tareas más frecuentes es extraer solo ciertos campos o columnas de una tabla. La instrucción SELECT en SQL permite especificar exactamente qué columnas queremos recuperar, evitando consultar todo el conjunto de datos, lo que es especialmente útil cuando se manejan grandes volúmenes de información.



Sintaxis básica de SELECT

```
SELECT  
    columna1,  
    columna2,  
    ...  
FROM nombre_tabla;
```

Este enfoque es útil para análisis, visualización de datos o generación de reportes parciales. La precisión en esta selección es clave en entornos productivos donde se requiere eficiencia en las consultas y protección de la información sensible.

Selección de todas las columnas

```
SELECT *  
FROM envios;
```

Esto resulta útil en etapas exploratorias o en tareas de depuración, pero no se recomienda en entornos controlados o productivos por razones de rendimiento, seguridad y claridad.

Buenas prácticas y errores comunes

Buenas prácticas

- Utiliza `SELECT *` solo si necesitas revisar toda la estructura de la tabla.
- En producción, declara explícitamente las columnas necesarias.
- Verifica que las columnas solicitadas sean relevantes para el propósito del análisis.

Errores comunes

- Recuperar más datos de los necesarios, aumentando el tiempo de respuesta y el consumo de recursos.
- Ocultar problemas de estructura (campos duplicados, sin normalizar) al usar `*` sin un propósito claro.

Uso de alias para mejorar la legibilidad

```
SELECT
    ciudad_destino AS ciudad,
    tipo_servicio AS servicio
FROM envios;
```

En contextos profesionales, especialmente al integrar datos en dashboards, reportes o tablas dinámicas, es común que los nombres de columnas en la base de datos no coincidan con el lenguaje de negocio. Para resolver esto, se pueden usar alias.

Esto mejora la comprensión de los resultados por parte de usuarios no técnicos y facilita la integración con herramientas de visualización.

Selección de columnas derivadas o calculadas

```
SELECT
    fecha_envio,
    estado,
    UPPER(ciudad_destino) AS ciudad_mayuscula
FROM envios;
```

SQL permite crear columnas nuevas a partir de transformaciones.

Esto permite personalizar la salida sin alterar la estructura de la tabla. Es útil cuando se quiere estandarizar resultados o preparar datos para su visualización.

Tipos de selección

Tipo de selección	Sintaxis ejemplo	Resultado esperado
Todas las columnas	SELECT * FROM envios;	Muestra toda la información, útil en exploración
Columnas específicas	SELECT estado, ciudad_destino FROM envios;	Muestra solo lo relevante para el análisis
Con alias	SELECT estado AS estado_envio FROM envios;	Renombra para mejorar claridad en reportes
Calculadas o derivadas	SELECT UPPER(ciudad_destino) AS ciudad_mayuscula ...	Personaliza o transforma valores directamente

Tabla 1. Comparativa entre tipos de selección en consultas SELECT.

Fuente: Desaío Latam

Aplicación práctica

Métrica clave

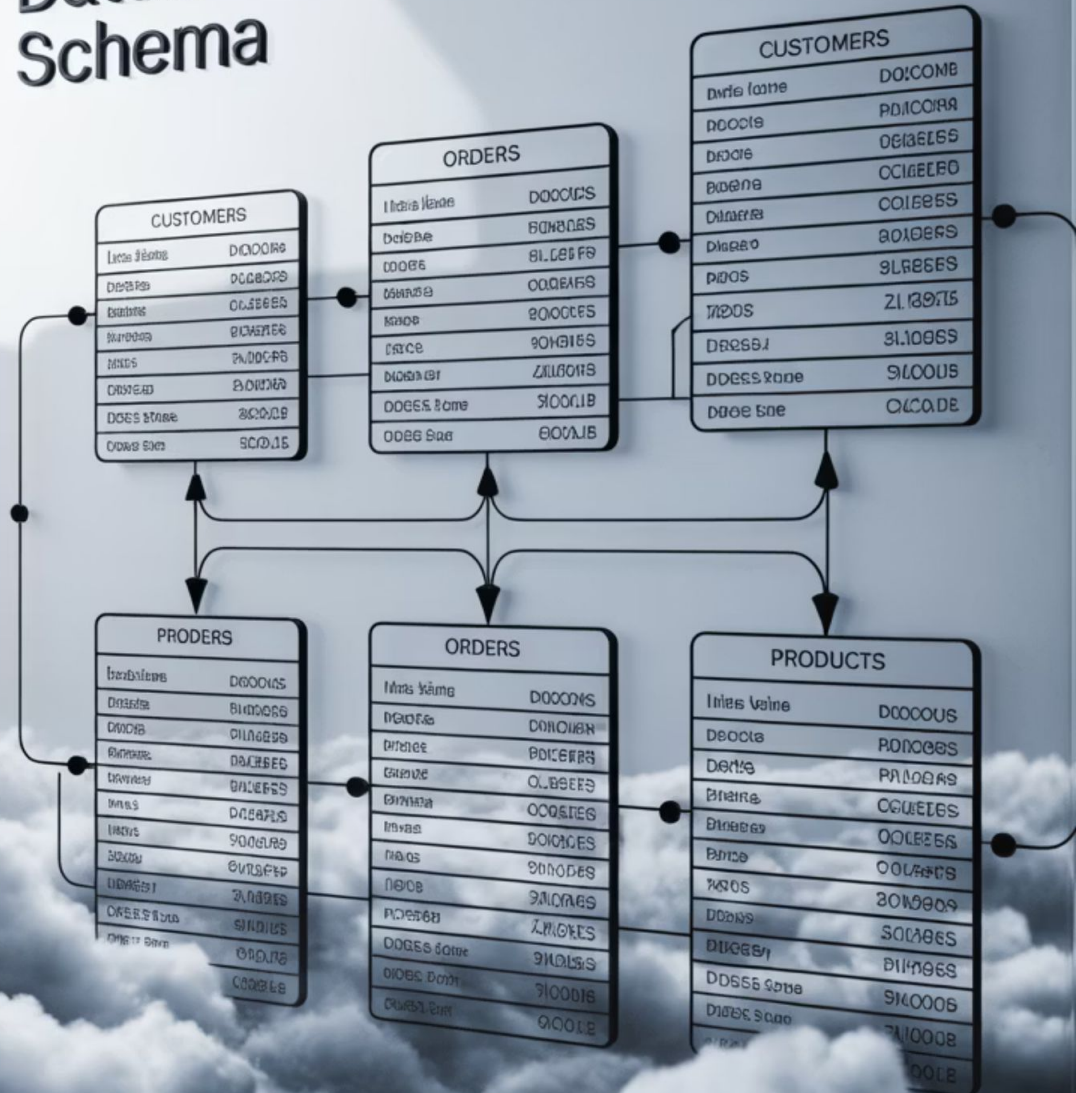
- Tamaño promedio de consulta: limitar el número de columnas mejora la velocidad y reduce el uso de memoria, especialmente en tablas con millones de registros.

Comprender cómo y por qué seleccionar columnas específicas no solo mejora el rendimiento, sino también la claridad y utilidad del análisis. Ahora que dominamos la selección de campos, necesitamos saber desde qué fuente (tabla) obtendremos esta información, lo que exploraremos a continuación mediante la cláusula FROM.

Aplicación en entornos reales

Un analista de datos en retail que debe calcular la facturación por tienda puede extraer solo fecha, tienda_id, y total_venta, ignorando decenas de otras columnas como descuento_promocional o tipo_pago, si estas no son relevantes para el objetivo del reporte.

Database Schema



Selección de tabla con FROM

¿De qué forma defines qué tabla es la fuente correcta para recuperar los datos que necesitas, especialmente cuando existen múltiples tablas similares o relacionadas?

Seleccionar la tabla adecuada desde la cual se recuperarán los datos es un paso fundamental en cualquier consulta SQL. La cláusula FROM permite especificar la tabla o conjunto de tablas desde donde se extraerán los registros. Una decisión incorrecta en esta etapa puede llevar a resultados erróneos, incompletos o redundantes, con consecuencias reales en la toma de decisiones empresariales.

Sintaxis básica de FROM

```
SELECT  
    columna1,  
    columna2  
FROM nombre_tabla;
```

Este fragmento indica a SQL desde qué tabla debe extraer los datos. La tabla debe existir en la base de datos y la persona analista debe tener los permisos necesarios para consultarla. Esta instrucción representa la conexión directa entre la estructura de la base de datos y el análisis solicitado.

Identificación de la tabla correcta en entornos reales

En bases de datos corporativas, existen múltiples tablas organizadas por dominio de negocio. En un sistema logístico, podría haber varias tablas: clientes, envios, rutas, transportistas. Si se quiere obtener el estado actual de los envíos, la tabla adecuada sería envios, pero si se requiere analizar las rutas históricas de transporte, probablemente se deba consultar rutas o envios_hist.

Buenas prácticas y errores comunes con FROM

Buenas prácticas

- Revisar la documentación del esquema de la base de datos.
- Utilizar herramientas de exploración de metadatos como pgAdmin, DBeaver, SQL Server Management Studio, entre otras.
- Realizar consultas exploratorias (LIMIT 10) para inspeccionar la calidad y granularidad de los datos.
- Validar que la tabla esté actualizada si es usada en contextos en tiempo real o reportes operativos.

Errores comunes

- Confundir tablas con nombres similares (envio, envios, envios_hist), lo que puede entregar resultados duplicados o incompletos.
- Ignorar claves primarias o relaciones, lo que puede llevar a consultas aisladas que omiten información crítica.
- Consultar tablas temporales o intermedias sin entender su contexto, provocando análisis erróneos.

Alias para tablas

```
SELECT
    e.fecha_envio,
    c.nombre_cliente
FROM envios AS e
JOIN clientes AS c ON e.cliente_id = c.id;
```

Cuando se trabaja con múltiples tablas, nombres largos o se requiere optimizar la legibilidad del código, se pueden utilizar alias. Esto también es indispensable al hacer joins entre tablas.

El uso de alias facilita la escritura, evita ambigüedades y mejora la trazabilidad de columnas cuando se hace referencia a múltiples tablas.

Verificación de estructura y registros previos

```
SELECT *  
FROM envios  
LIMIT 10;
```

Antes de construir consultas complejas, es útil obtener una vista previa del contenido.

Esto permite validar nombres de columnas, tipos de datos y verificar si los datos están completos, lo cual es esencial para prevenir errores en consultas más avanzadas.

Esquemas y prefijos de tabla

```
SELECT *  
FROM produccion.envios;
```

En sistemas grandes, las tablas pueden estar organizadas por esquemas.

Aquí, produccion es el esquema que contiene la tabla envios. Usar el esquema correcto garantiza que se extraen datos desde el entorno apropiado (producción, pruebas, respaldo, etc.).

Usos comunes de FROM en diferentes contextos

Contexto productivo	Tabla de origen típica	Objetivo de consulta
Gestión académica	estudiantes	Obtener listado de personas matriculadas
Logística	envios	Consultar estados y fechas de despacho
Comercio electrónico	ordenes	Analizar ventas por período
Atención al cliente	tickets_soporte	Medir tiempos de respuesta
Recursos humanos	asistencias, nominas	Calcular horas trabajadas y pagos

Tabla 2. Ejemplos de uso de la cláusula FROM en distintos sectores.

Fuente: Desafío Latam

Aplicación práctica de FROM

Aplicación en entornos reales

En una empresa de telecomunicaciones, un analista necesita extraer la facturación mensual por cliente. Si confunde la tabla `facturacion_detalle` con `facturacion_resumen`, podría obtener datos duplicados o incompletos, afectando informes financieros. Elegir correctamente la fuente asegura integridad y relevancia de los datos.

Seleccionar correctamente la tabla desde la cual extraer datos es esencial para garantizar precisión y confiabilidad en los análisis. Ahora que dominamos las sentencias `SELECT` y `FROM`, el paso siguiente es filtrar los resultados para que solo se incluyan los registros relevantes. Esto se logra mediante el uso de la cláusula `WHERE`, que abordaremos a continuación.

Métrica clave

- Consistencia referencial: la tabla seleccionada debe integrarse correctamente con otras fuentes mediante claves externas, garantizando que el análisis no esté basado en datos huérfanos o inconexos.



Filtrado básico con WHERE

¿Cómo puedes restringir los resultados de una consulta para que solo devuelvan los registros que realmente necesitas?

La cláusula WHERE en SQL permite aplicar condiciones a una consulta para filtrar los resultados. Esta capacidad es clave cuando se trabaja con bases de datos grandes o complejas, donde extraer todos los registros no solo es ineficiente, sino también innecesario o incluso riesgoso para el negocio.

Sintaxis básica del uso de WHERE

```
SELECT  
    columna1,  
    columna2  
FROM nombre_tabla  
WHERE condición;
```

Por ejemplo, si se quiere obtener solo los envíos que están "en tránsito":

```
SELECT  
    id_envio,  
    estado  
FROM envios  
WHERE estado = 'En tránsito';
```

Esta cláusula puede combinarse con múltiples operadores y condiciones para refinar la consulta.

Uso de operadores de comparación

Operador	Significado	Ejemplo
=	Igual a	estado = 'Entregado'
<> o !=	Distinto de	estado <> 'Pendiente'
>, <	Mayor o menor que	peso > 10
>=, <=	Mayor o igual, etc.	fecha_envio <= '2024-12-31'

Tabla 3. Uso de operadores de comparación

Fuente: Desafío Latam

Estas comparaciones permiten aplicar filtros precisos. En un entorno de análisis de ventas, por ejemplo, filtrar por total_venta >= 100000 ayuda a identificar operaciones relevantes.

Combinación de condiciones: AND, OR y NOT

```
SELECT *  
FROM envios  
WHERE estado = 'En tránsito'  
      AND ciudad_destino = 'Valparaíso';
```

Con AND se requieren que ambas condiciones se cumplan.

Con OR, basta con que una lo haga:

```
SELECT *  
FROM envios  
WHERE ciudad_destino = 'Iquique'  
      OR ciudad_destino = 'Arica';
```

Y con NOT se puede excluir explícitamente valores:

```
SELECT *  
FROM envios  
WHERE estado != 'Cancelado';
```


Filtrado con rangos: BETWEEN y fechas

```
SELECT *  
FROM envios  
WHERE fecha_envio BETWEEN '2024-06-01' AND '2024-06-30';
```

BETWEEN permite especificar rangos de valores, útil para fechas, precios o cantidades.

Filtrado por listas: IN

```
SELECT *  
FROM envios  
WHERE ciudad_destino IN ('Santiago', 'Valparaíso', 'Concepción');
```

Evita escribir múltiples OR y mejora la legibilidad. También se puede usar con subconsultas.

Filtrado por texto parcial: LIKE y comodines

```
SELECT *
FROM clientes
WHERE nombre_cliente LIKE 'Mar%';
```

El operador LIKE permite buscar patrones. % representa cualquier número de caracteres, _ un solo carácter.

Comparación de métodos de filtrado con WHERE

Técnica	Sintaxis ejemplo	Uso recomendado
Igualdad	WHERE estado = 'Pendiente'	Filtrar por estado exacto
Rango	WHERE total_venta BETWEEN 1000 AND 5000	Valores dentro de un rango
Lista (IN)	WHERE ciudad IN ('Arica', 'Iquique')	Varios valores específicos
Texto parcial (LIKE)	WHERE nombre LIKE 'Ana%'	Búsquedas por coincidencia parcial
Combinación lógica	WHERE estado = 'En tránsito' AND ciudad = 'Valparaíso'	Condiciones múltiples

Tabla 3. Métodos de filtrado de datos con la cláusula WHERE.

Fuente: Desafío Latam

Errores comunes y buenas prácticas con WHERE

Errores comunes

- Usar comillas simples en números: WHERE total = '1000' (incorrecto).
- Omitir condiciones clave y obtener resultados inesperados o demasiado amplios.
- Confundir el uso de LIKE con operadores de igualdad, causando consultas ineficientes.

Buenas prácticas

- Filtrar lo antes posible para reducir la carga del sistema.
- Validar que los campos filtrados tengan índices si se trabaja con bases de datos grandes.
- Documentar las condiciones para trazabilidad de análisis.

Aplicación práctica de WHERE

Aplicación en entornos reales

En un sistema de ventas regional, un analista necesita detectar ventas de alto valor realizadas en una zona específica en los últimos 30 días.

La consulta SQL con WHERE permitirá recuperar únicamente esos registros, facilitando alertas tempranas, análisis de desempeño o auditoría.

La cláusula WHERE es una herramienta poderosa para delimitar y optimizar los resultados de una consulta. Dominar su uso permite responder con agilidad a necesidades analíticas concretas. En la siguiente actividad guiada, pondrás en práctica estos conceptos para resolver un caso real, conectando los aprendizajes de esta sesión con su aplicación en un entorno productivo.

Métrica clave

- Eficiencia de consulta: al reducir la cantidad de registros procesados, se minimizan tiempos de ejecución y uso de recursos, mejorando la experiencia del usuario final.

Actividad guiada:

**Consultas básicas para
optimizar el monitoreo
logístico**



Objetivo de la actividad

Aplicar consultas SQL básicas utilizando las sentencias SELECT, FROM y WHERE para extraer información relevante desde una base de datos de envíos, considerando criterios múltiples como estado del envío, destino y fecha, en el contexto de análisis operativo logístico.



Situación problema

Trabajas en el área de análisis de datos de una empresa de logística nacional. La gerencia ha detectado demoras en ciertas regiones y te han solicitado preparar un reporte que muestre solo los envíos que están actualmente en tránsito hacia ciudades específicas y que hayan sido enviados durante los últimos 30 días.

La base de datos contiene muchos registros históricos, por lo que debes aplicar filtros precisos para evitar errores o sobrecarga en el sistema.



Instrucciones

Crear la tabla envios simulada con los siguientes datos:

```
CREATE TABLE envios (  
  id_envio INT,  
  ciudad_destino TEXT,  
  estado TEXT,  
  fecha_envio DATE  
);  
  
INSERT INTO envios VALUES  
  (1, 'Valparaíso', 'En tránsito', '2024-07-01'),  
  (2, 'Santiago', 'Entregado', '2024-06-20'),  
  (3, 'Iquique', 'En tránsito', '2024-07-10'),  
  (4, 'Valparaíso', 'Cancelado', '2024-07-05'),  
  (5, 'Arica', 'En tránsito', '2024-06-29');
```



Instrucciones

Consultar todos los registros:

```
SELECT * FROM envios;
```

Insertar datos de prueba:

```
SELECT
  id_envio,
  ciudad_destino,
  estado,
  fecha_envio
FROM envios
WHERE estado = 'En tránsito'
  AND ciudad_destino IN ('Valparaíso', 'Iquique')
  AND fecha_envio >= '2024-07-01';
```

Resumen

1

SELECT

Aprendimos a utilizar SELECT para elegir columnas específicas y optimizar consultas.

2

FROM

Comprendimos la función de FROM para indicar las tablas fuente.

3

WHERE

Aplicamos WHERE para filtrar resultados según múltiples condiciones.

4

Buenas prácticas

Analizamos buenas prácticas y errores comunes en consultas SQL básicas.

5

Aplicación

Conectamos la sintaxis SQL con aplicaciones en entornos productivos reales.



Próxima sesión...

En la siguiente sesión profundizaremos en el uso de operadores y condiciones dentro de las cláusulas WHERE.

Aprenderemos a utilizar operadores condicionales, sentencias como IN, NOT IN y BETWEEN, así como los operadores lógicos AND y OR, para realizar filtrados más complejos y flexibles.

Esto permitirá construir consultas más precisas y adaptadas a necesidades reales de análisis en bases de datos.

Preguntas de cierre



¿Qué diferencias existen entre seleccionar todas las columnas con * y definir columnas específicas con SELECT?



¿Qué criterios debes considerar para decidir desde qué tabla extraer los datos?



¿Cómo se combinan múltiples condiciones dentro de una cláusula WHERE?



¿Qué problemas podrías enfrentar si omites filtros en tus consultas?



¿Por qué es importante comprender el tipo de datos de cada campo al aplicar condiciones?

Referencias bibliográficas

- Coronel, C., & Morris, S. (2019). Database Systems: Design, Implementation, & Management (13th ed.). Cengage Learning.
- Melton, J., & Simon, A. R. (2002). Understanding the New SQL: A Complete Guide. Morgan Kaufmann.
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/>
- W3Schools. (2024). SQL Tutorial. <https://www.w3schools.com/sql/>